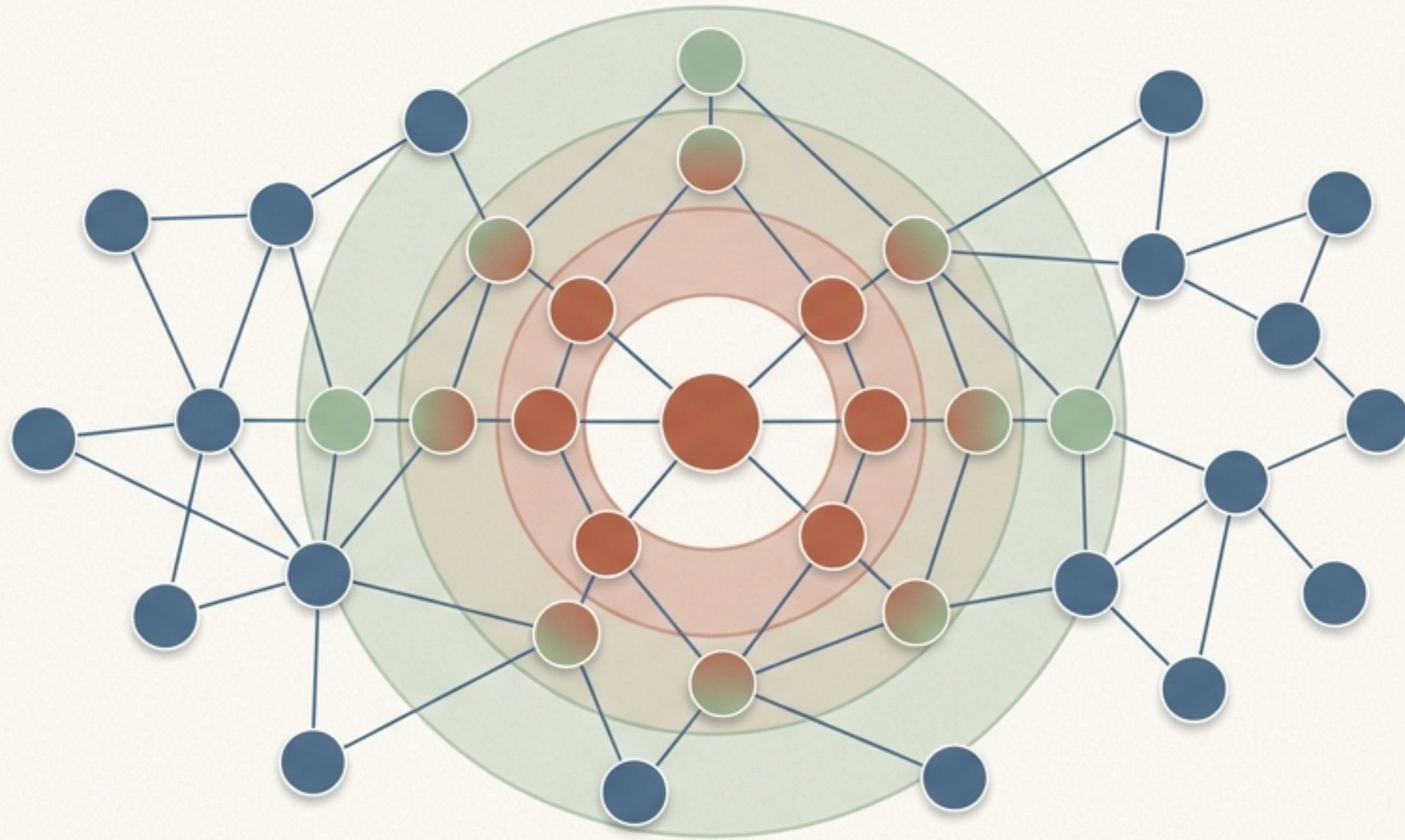
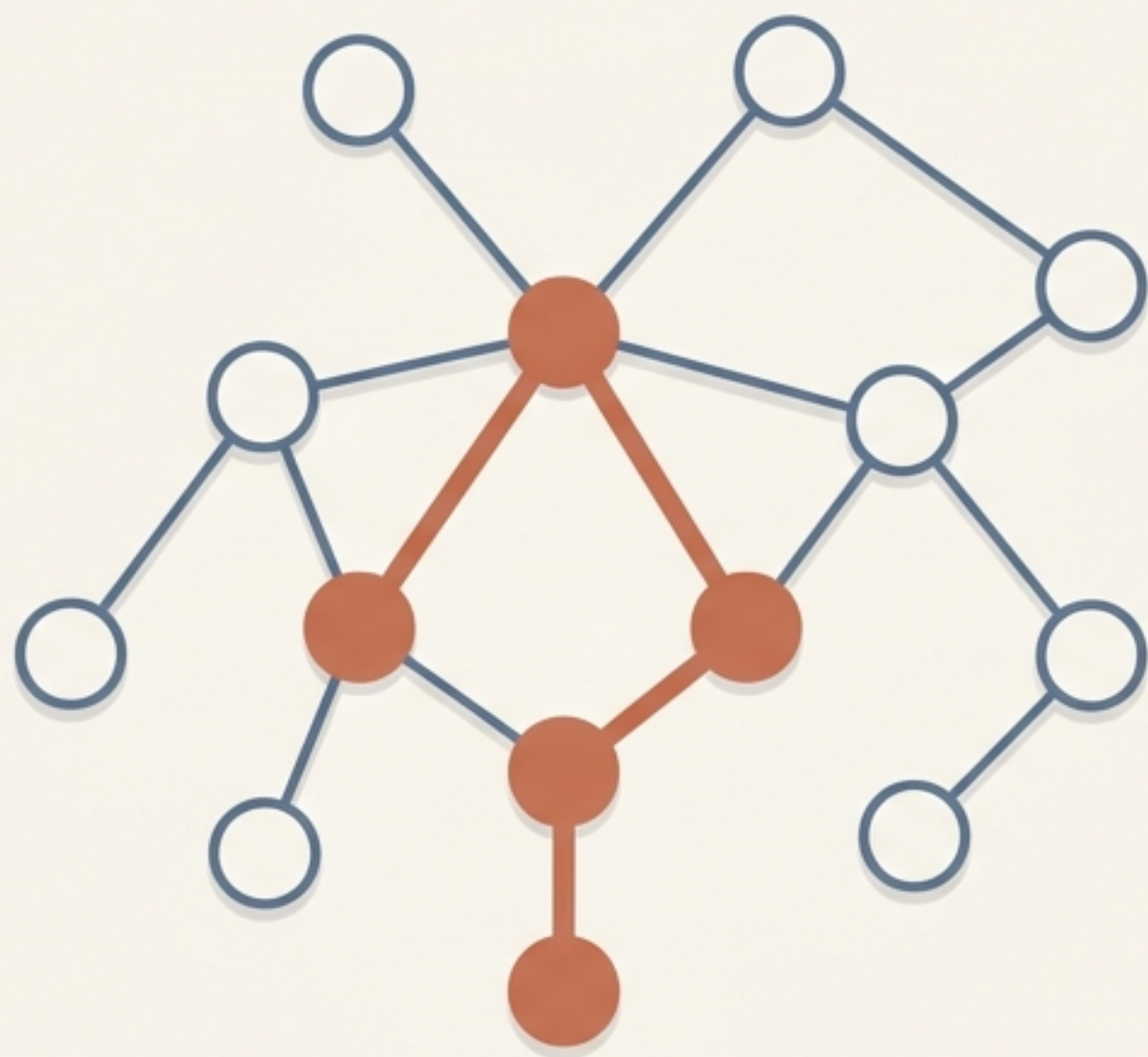


Mastering Breadth-First Search

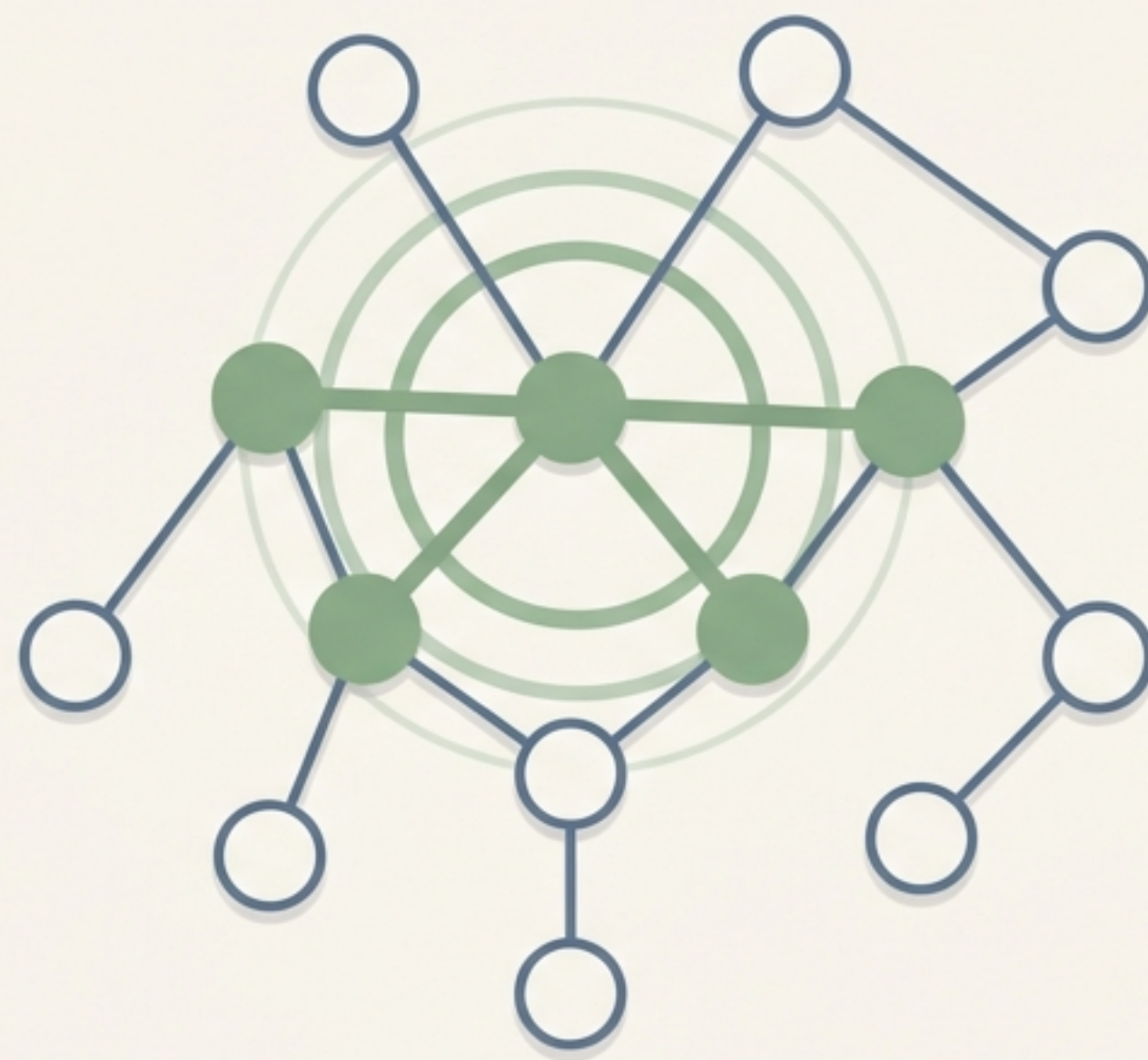
From Conceptual Mental Models to Python Implementation





Depth-First Search (DFS):

Explores as deep as possible along one path before backtracking. Ideal for finding a single valid path to a destination.

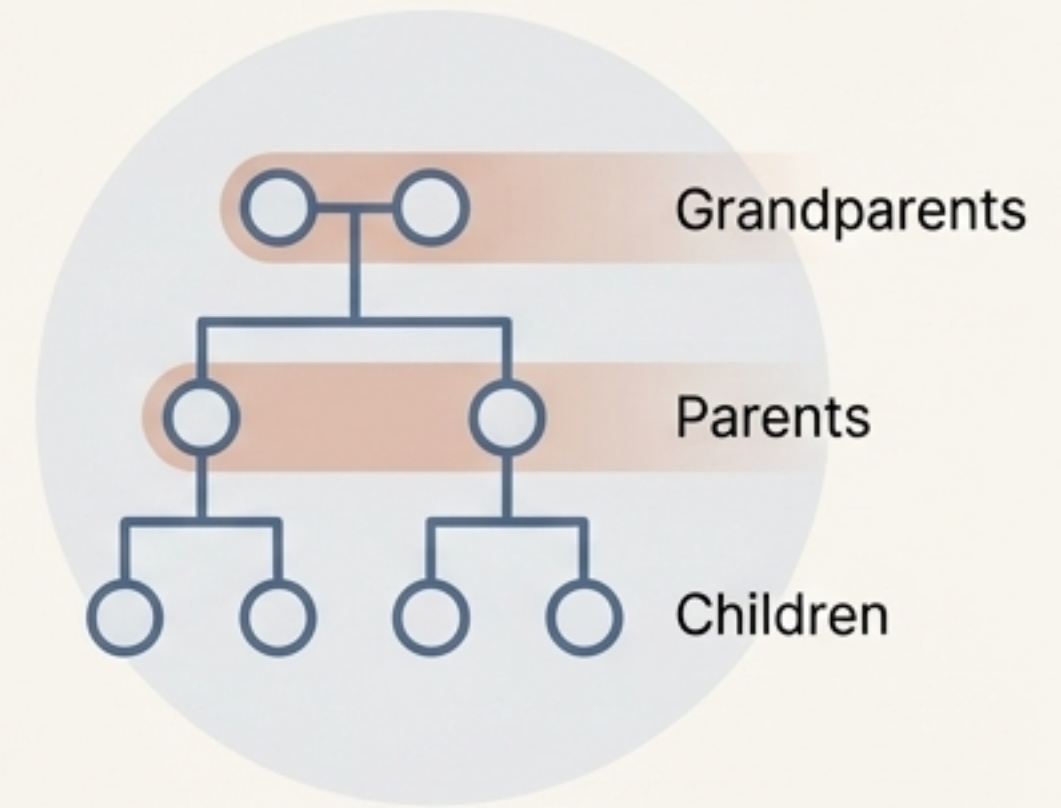
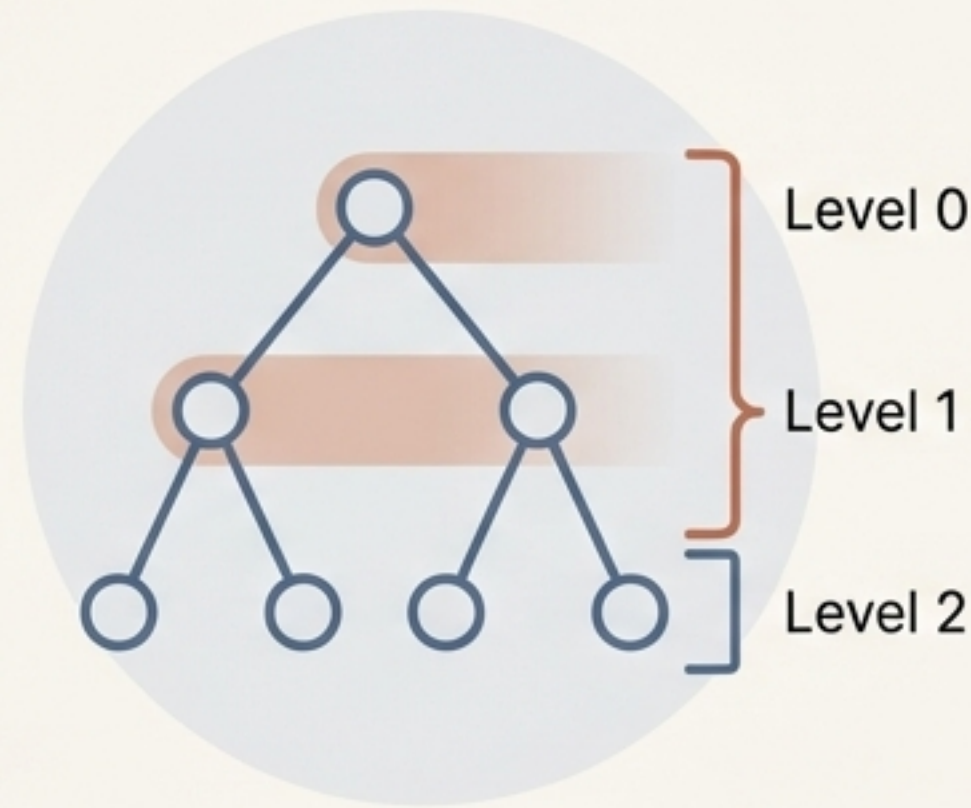
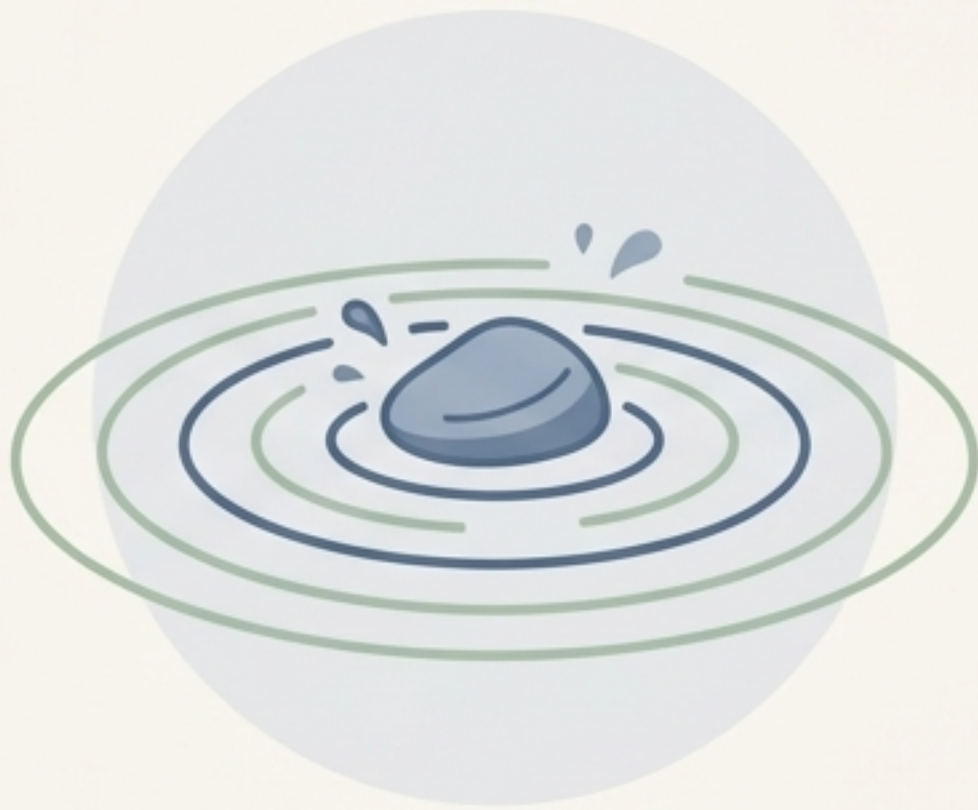


Breadth-First Search (BFS):

Explores all neighboring nodes at the present depth level before moving deeper. Ideal for level-order traversal and shortest-path scenarios.

The Mental Models of BFS

Think of BFS as an exploration of levels. You only move to the next phase of depth once all options in the current phase are completely exhausted. The breadth expands outward sequentially, generation by generation.

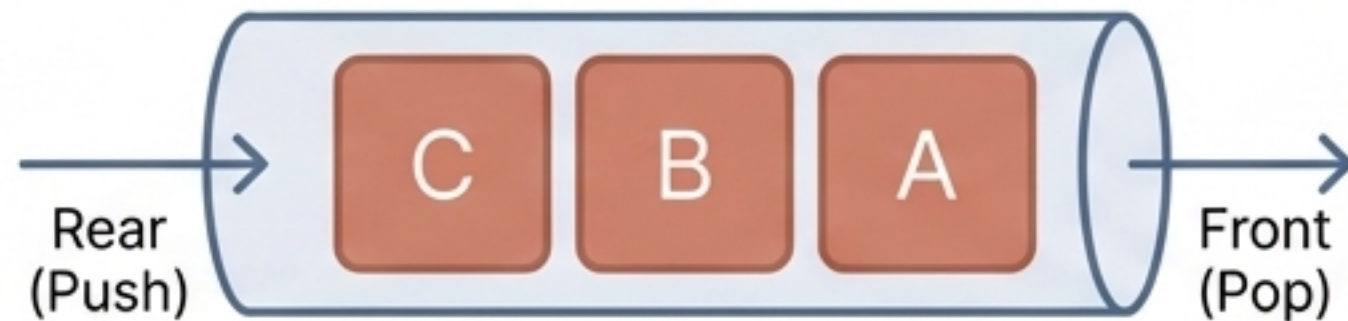


The Engine: Tools Required for BFS

To manage traversal order and prevent infinite loops, BFS relies on two essential data structures:

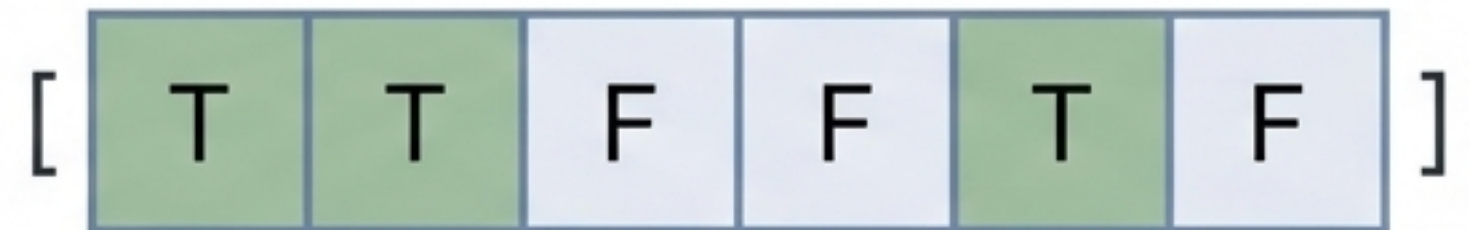
The Queue

Tracks which node to visit next based on a First-In, First-Out rule.



The Visited Array

Tracks which nodes have already been processed to avoid redundant loops.



The BFS Algorithm in 3 Steps

Step 1. Initialize:

Push the starting node into the **Queue** and mark it **True** in the **Visited array**.

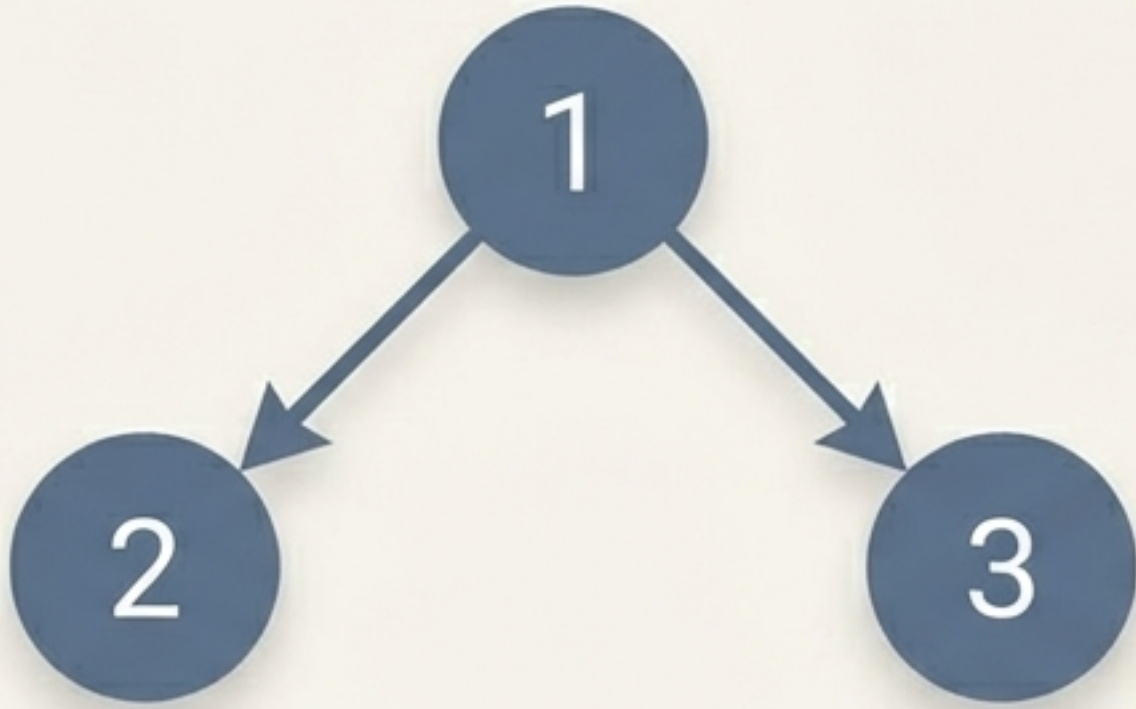
Step 2. Pop & Print:

While the Queue is not empty, remove the **front node** and output it.

Step 3. Explore Neighbors:

Find all unvisited, directly connected **neighbors**. Mark them **True** and append them to the back of the Queue. Loop back to Step 2.

Important Rule: No Single Right Answer



Output A:

[1, 2, 3]



Output B:

[1, 3, 2]

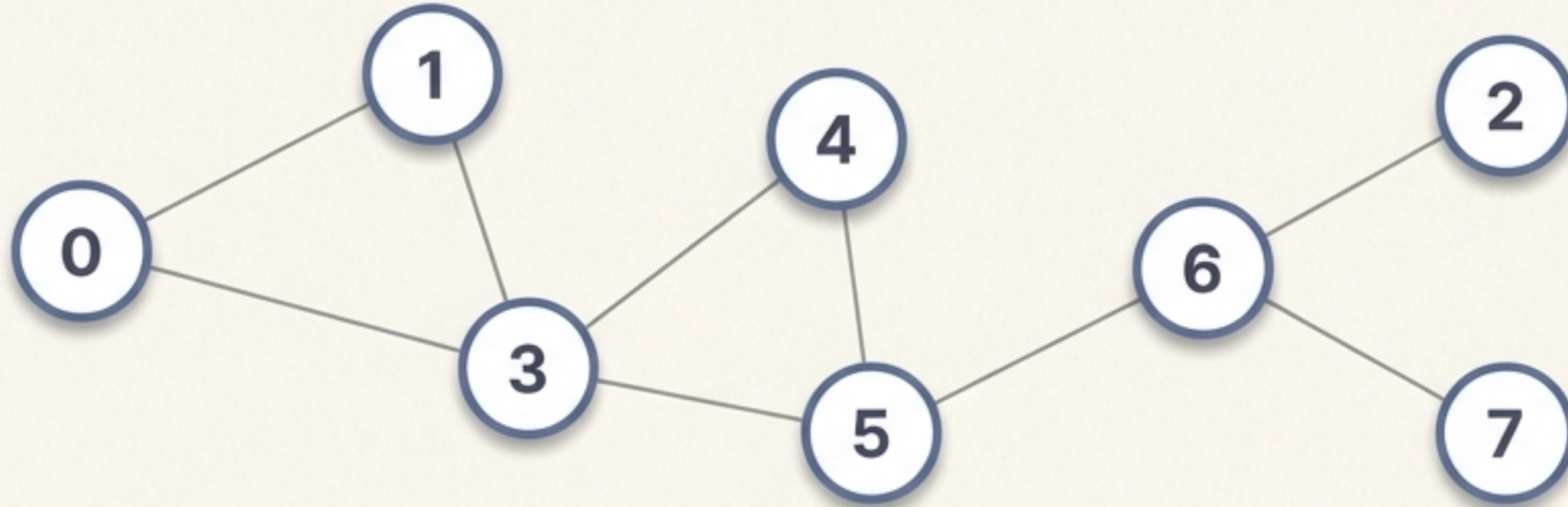


Because graphs do not have a strict Left or Right direction, neighbors can be added to the queue in any order. Nodes within the same level can be swapped in the final output, and the BFS execution remains 100% correct.

Visual Walkthrough: The Setup

Let's dry-run an 8-vertex graph starting at Node 0. We initialize our **Visited** array to size 8, populated with **False**.

Top Band
(Graph)

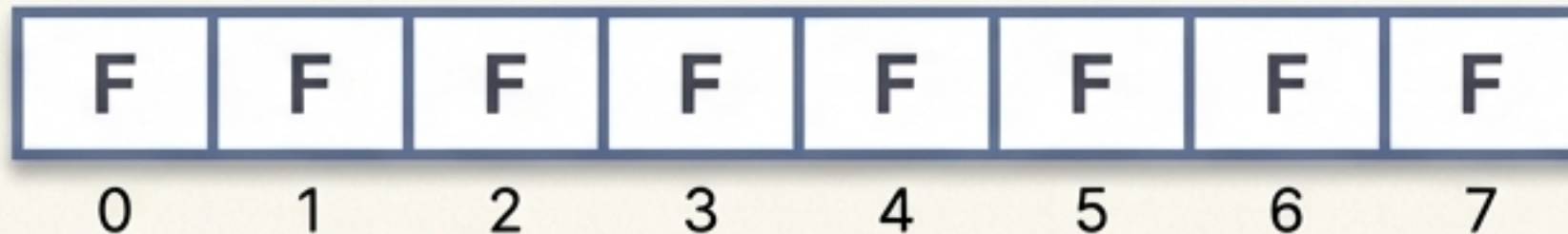


Middle Band
(Queue)

Queue: []

Bottom Band
(State)

Visited Array:

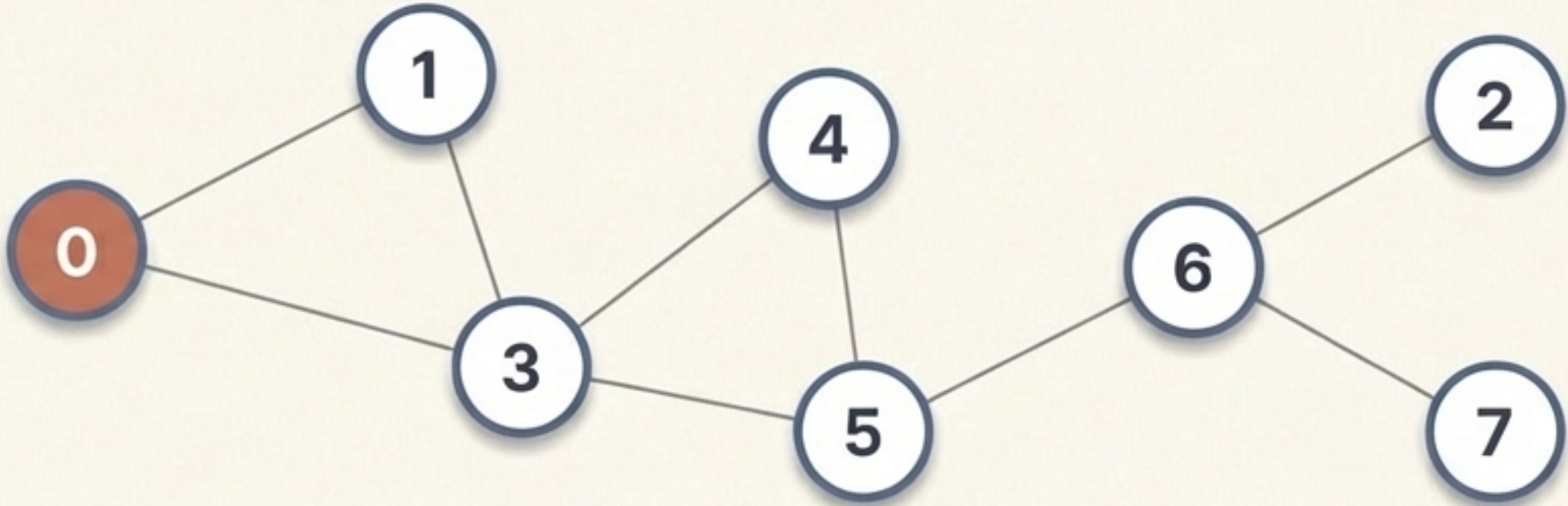


Current Output: []

Level 1 (The Source)

We push the starting node (**0**) into the **Queue** and immediately mark it as **Visited** so it is never processed again.

Top Band
(Graph)



Middle Band
(Queue)

Queue: [0]

Bottom Band
(State)

Visited Array:

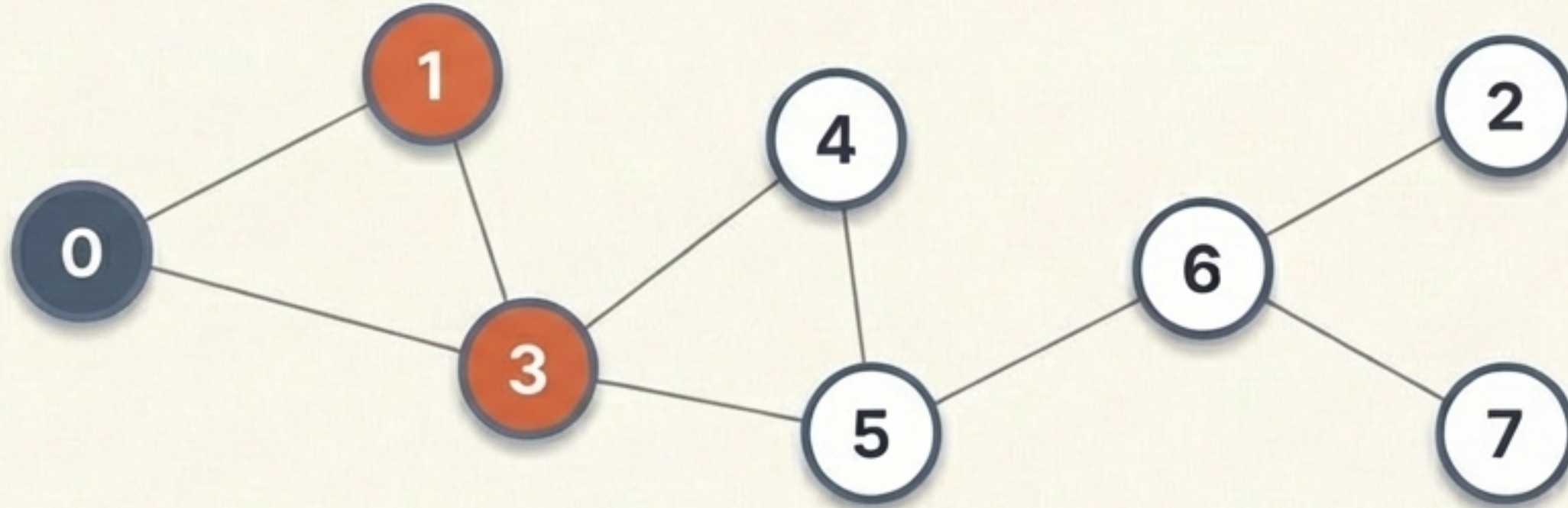
T	F	F	F	F	F	F	F
0	1	2	3	4	5	6	7

Current Output: []

Level 2 (The First Wave)

Pop **0** and print it. Find its unvisited neighbors (**1** and **3**). Mark them **True** and append them to the back of the **Queue**.

Top Band
(Graph)

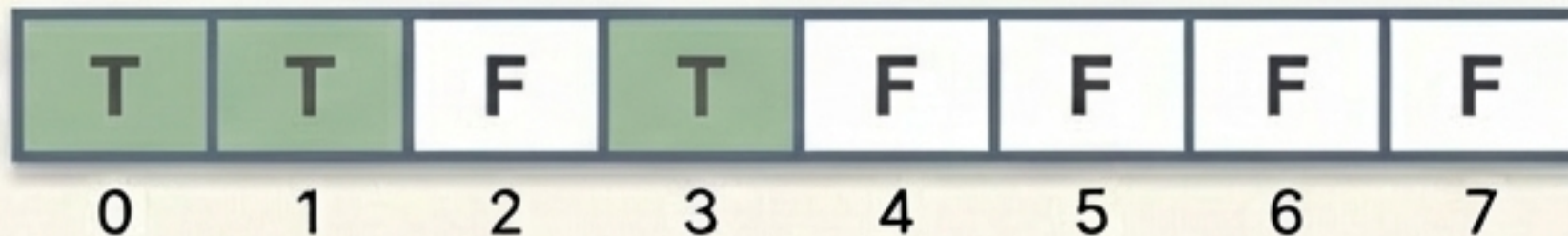


Middle Band
(Queue)

Queue: 1, 3

Bottom Band
(State)

Visited Array:

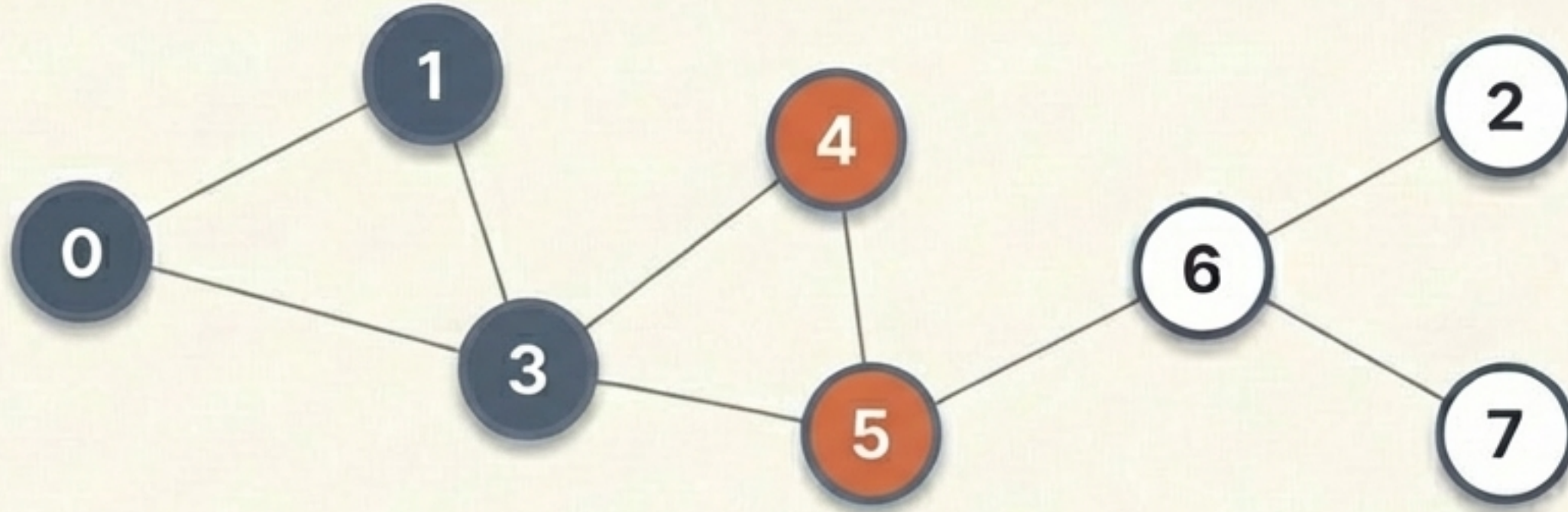


Current Output: [0]

Level 3 (Expanding the Breadth)

Process the queue sequentially. Node **1** has no unvisited neighbors. Node **3** connects to **4** and **5**. We mark **4** and **5 True** and append them. Notice how all Level 2 nodes finish before Level 3 nodes are queued.

Top Band
(Graph)

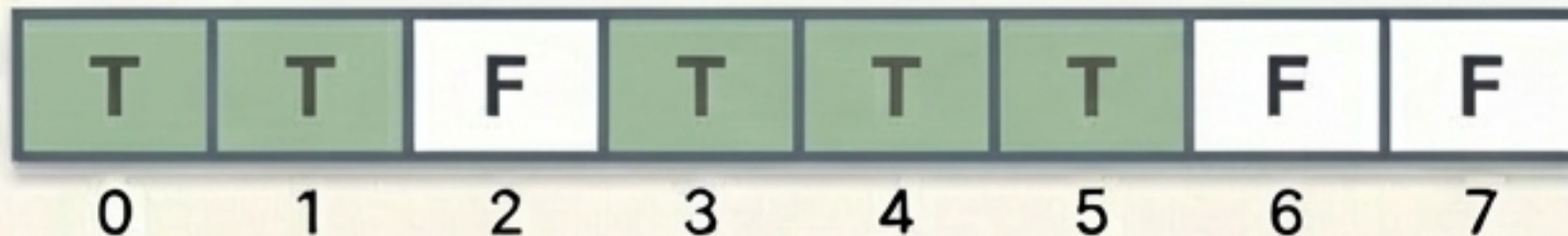


Middle Band
(Queue)

Queue: 4, 5

Bottom Band
(State)

Visited Array:

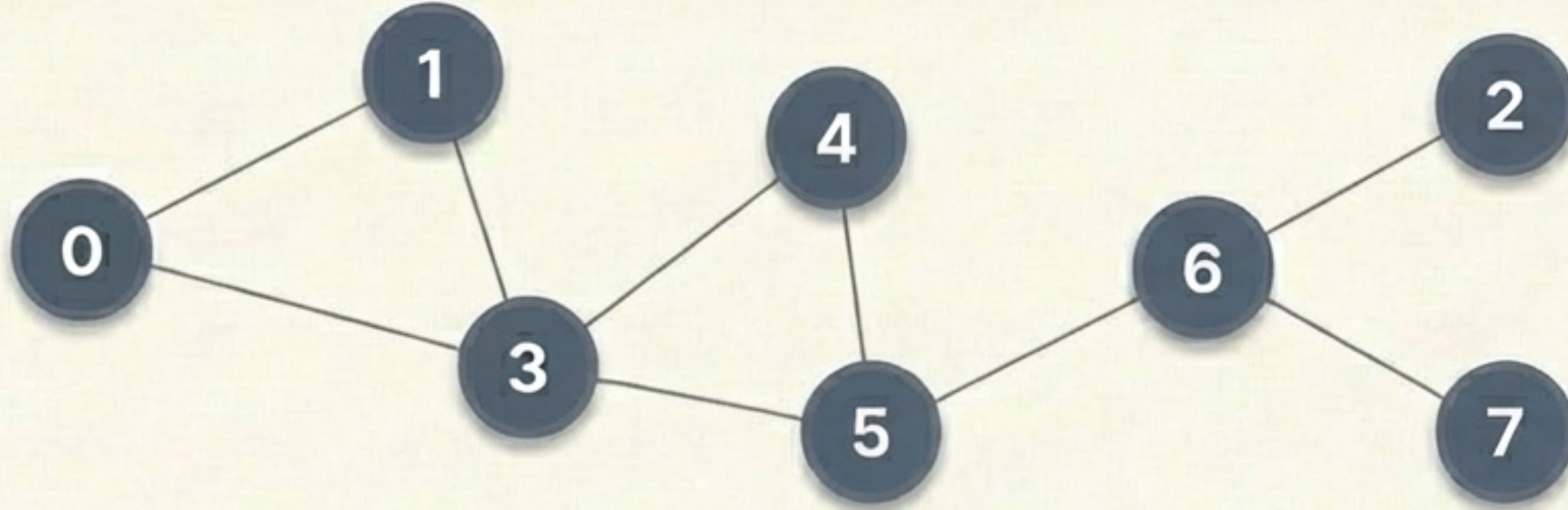


Current Output: [0, 1, 3]

Finishing the Traversal

We continue popping from the left and appending to the right. Once Node 7 is processed, no unvisited neighbors remain. The Queue becomes empty, and the execution terminates.

**Top Band
(Graph)**

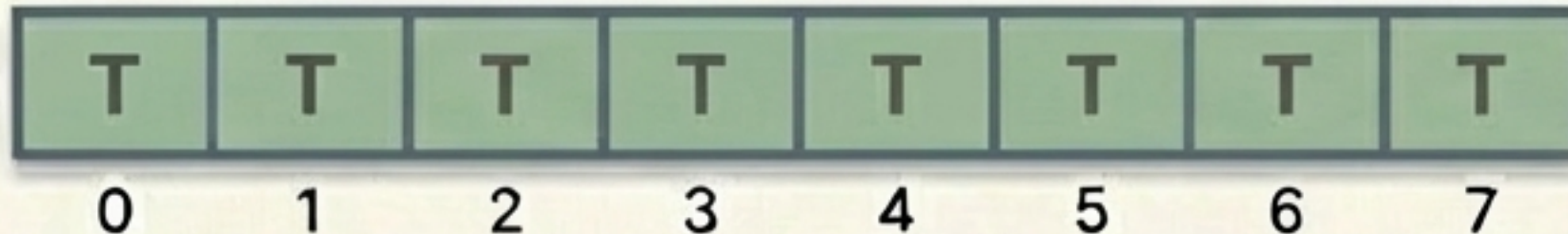


**Middle Band
(Queue)**

Queue: []

**Bottom Band
(State)**

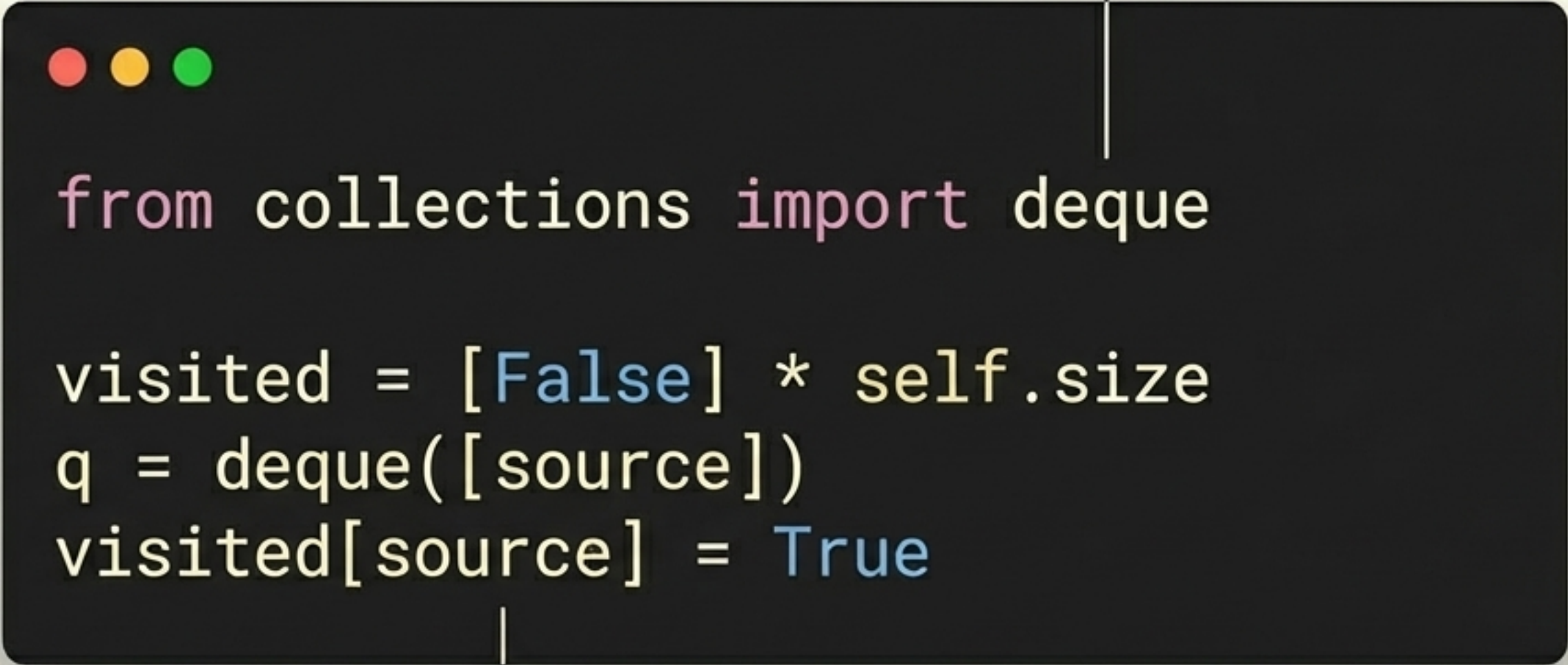
Visited Array:



Current Output: [0, 1, 3, 4, 5, 6, 2, 7]

Python Implementation: The Setup

1. Use **collections.deque** for highly efficient $O(1)$ pop operations from the left side of the queue.



```
from collections import deque

visited = [False] * self.size
q = deque([source])
visited[source] = True
```

2. Pre-allocates the visited tracking array based on the total number of graph vertices.

Python Implementation: The Engine

Extracts the active node to evaluate.

```
while q:
    v = q.popleft()
    print(v, end=' ')
    for i in range(self.size):
        if matrix[v][i] == 1 and not visited[i]:
            visited[i] = True
            q.append(i)
```

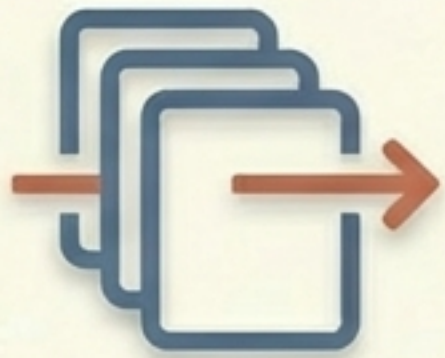
Iterates through the Adjacency Matrix to find valid edges (1).

Ensures we never queue a node we've already seen.

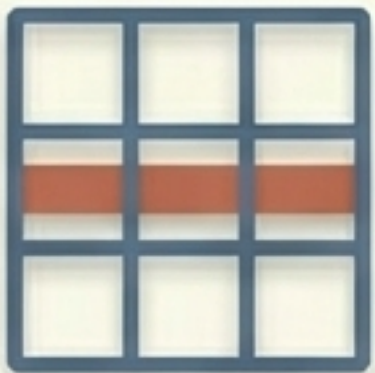
Core Takeaways



Level-Order Guaranteed: BFS explores radially. You will never visit a deeper node before entirely exhausting the current depth level.



Queue-Driven: The First-In, First-Out (FIFO) nature of a Queue is the mechanical engine that enforces breadth over depth.



Matrix Traversal: In an Adjacency Matrix implementation, finding neighbors requires scanning the active node's **entire row**, resulting in an $O(V^2)$ time complexity.